

盘古训练与推理通信优化实践： 亲和昇腾平台的性能探索

演讲人：李柏潮

华为 / 大模型系统工程通信技术专家

AiCon
全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon 上海站
全球软件开发大会

大模型正在重新定义软件
Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon 北京站
全球人工智能开发与应用大会

探索 AI 应用边界
Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

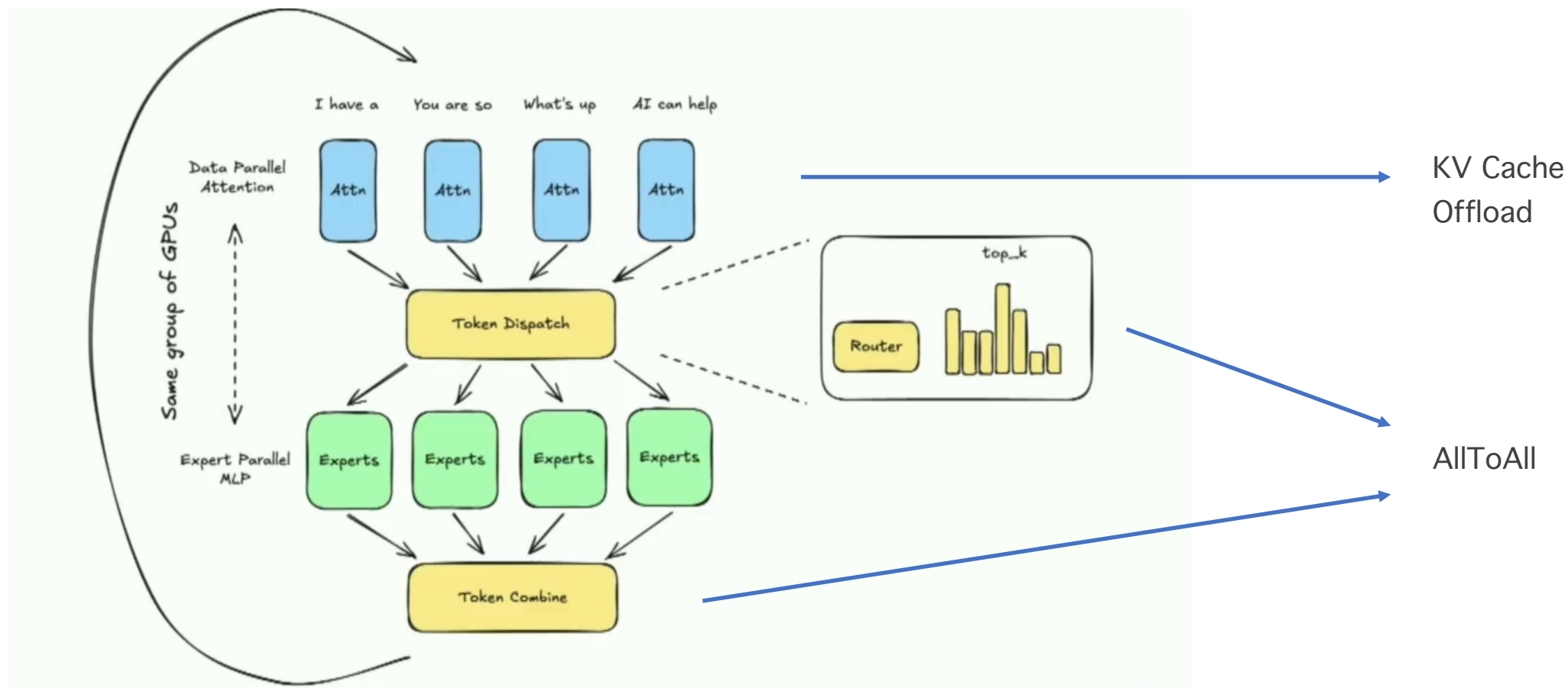
目录

- 01 大模型训练/推理在通信方面面临两大技术挑战
- 02 亲和昇腾硬件，加速 EP 域的AllToAll通信
- 03 硬件软件双管齐下，加速H2D KV Cache加载
- 04 总结和展望：减少未被掩盖的通信时间

01

大模型训推在通信方面 两大挑战

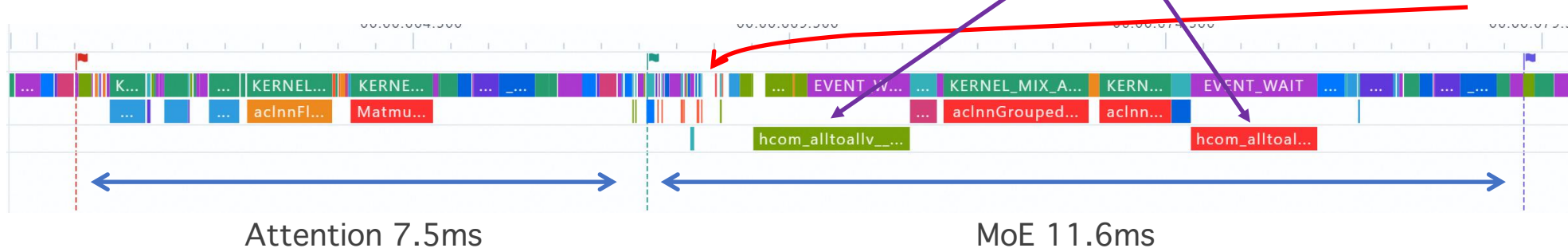
■ 背景：MoE模型中KV Cache Offload和EP AllToAll是通信最关键的两大挑战



挑战1: EP域AllToAll是大模型训练中的传统瓶颈

以盘古模型训练为例，MoE阶段占一个layer的forward计算时间的60%，其中两次AllToAllV合计4.46ms，占总时间的23%

另外, AllToAllV之前需要 AllGather 收集 top k 信息, Host 计算 AllToAllV 的 shape 才能下发算子, 造成算子断流300us



业界提出了非常多方案来优化AllToAllV的时间，其中最为典型是两个：

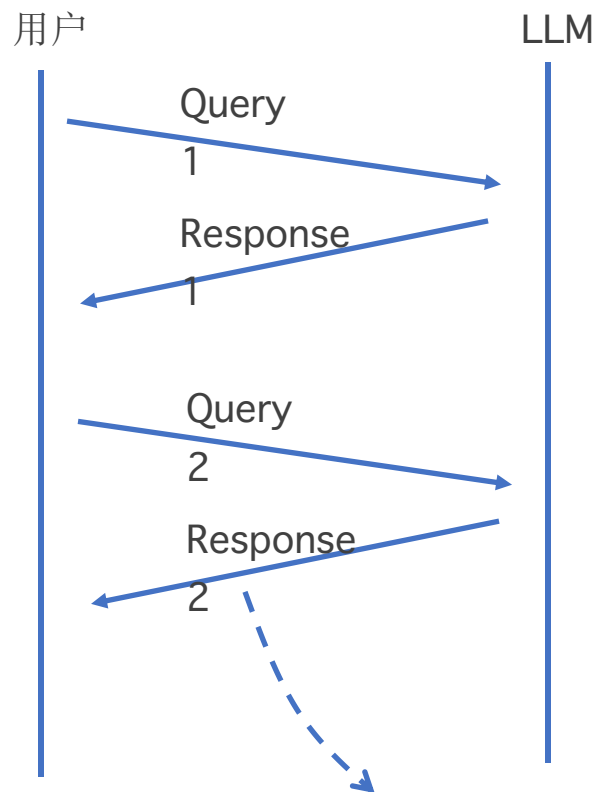
- DeepSeek 提出了 DeepEP 方案，实现跨 Server 时一个卡作为代理中转，加速跨 Server 通信
- PyTorch 结合 NVIDIA 的对称内存方案提出了 `all_to_all_vdev_2d` 减少了内存拷贝，实现 Device 驱动，减少 Host 下发时间

在昇腾平台上，我们有更优的解决方案



挑战2：1M上下文推理场景，KV Offload 成为必选项

在Agent场景，用户和模型会进行多轮交互。第N轮交互计算Attention时，需要把第1轮到第N-1轮的所有KV Cache一起计算，造成KV Cache 占用巨大的HBM空间



- 对于DeepSeek V3，使用 MLA，单个 1M 上下文序列需要 70 GB的 KVCache
- 对于盘古模型（openPangu-2.0-Pro），使用 MLA+DSA 混合架构，最大支持512k 上下文长度，需要 35 GB的 KVCache
- 对于DeepSeek V4，在 MLA 的基础上引入了 Compressed Attention（包含 CSA 4x 窗口压缩与 HCA 128x 深度压缩），1M上下文需要7GB

所以，历史对话的 KV Cache 不能一直占用 HBM 空间，需要卸载到 DDR 上。

在昇腾平台如何实现高效的 KV Cache 加载和卸载呢？

Prompt = Query 1 + Response 1 + Query 2

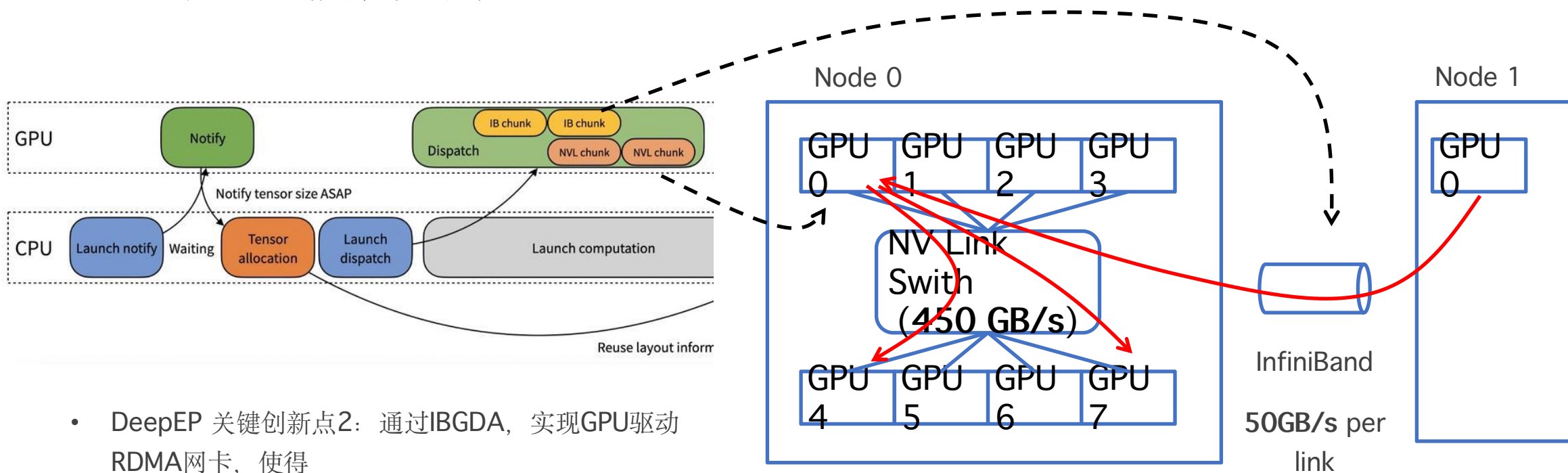
02

亲和昇腾硬件， 加速 EP 域的 AI To AI 通信



拓扑亲和：DeepEP适合NV拓扑，不适合昇腾超节点

- DeepEP 关键创新点1：针对NVIDIA服务器拓扑，适应 Node 内 NVLink 大带宽，Node 间 InfiniBand 低带宽，进行了机内和机间的流水，NV chunk收集机内信息（完成机内通信，收集跨机通信数据），IB chunk进行跨机同号卡交互，仅同号卡交互，代理发到其他GPU的数据，减少了跨机RDMA连接



- DeepEP 关键创新点2：通过IBGDA，实现GPU驱动RDMA网卡，使得

■ 昇腾超节点实现大范围（A3是384卡）高速互连，应该使用Mesh拓扑，不需要分级通信

- Atlas 900 A3 SuperPoD 实现384卡通过HCCS高速互连，Server内和Server间带宽1:1无收敛，不需要进行分级直通，应该直接使用Mesh算法进行AllToAll
- 注：Atlas 800T A3超节点服务器支持HCCS互连和RoCE互连，如果使用RoCE互连，应该参照DeepEP方案进行分级通信
- 超节点内可以实现总线访问，所有卡的HBM进行统一编址，Vector通过 load / store 内存语义读写数据，原生实现NPU驱动通信



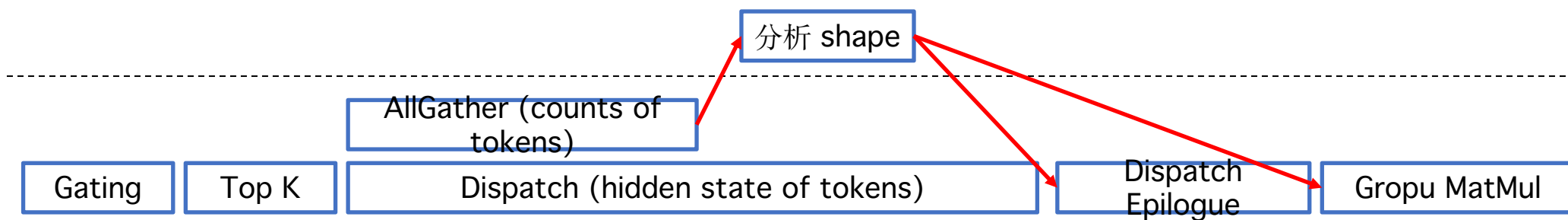
超节点架构

- 最大支持384*NPU高速互联，D2D双向带宽784GB/s
- 48TB片上内存统一编址
- 总线带宽1:1无收敛
- 200ns通信单跳时延

■ 模型训练场景，亲和昇腾拓扑的Dispatch/Combine

技术关键点有两个：

- 根据训练的batch size预留接收缓冲区（最坏情况下所有DP域的token都需要发给一个专家），各个rank在计算top k之后就可以直接按照预定义好的地址偏移，使用Mesh算法，立刻启动数据发送
- 在数据发送的过程中，进行All Gather收集各个rank的top k计算结果，host计算后续GMM的shape，下发GMM算子。Host计算时间和算子下发时间被Dispatch时间掩盖。



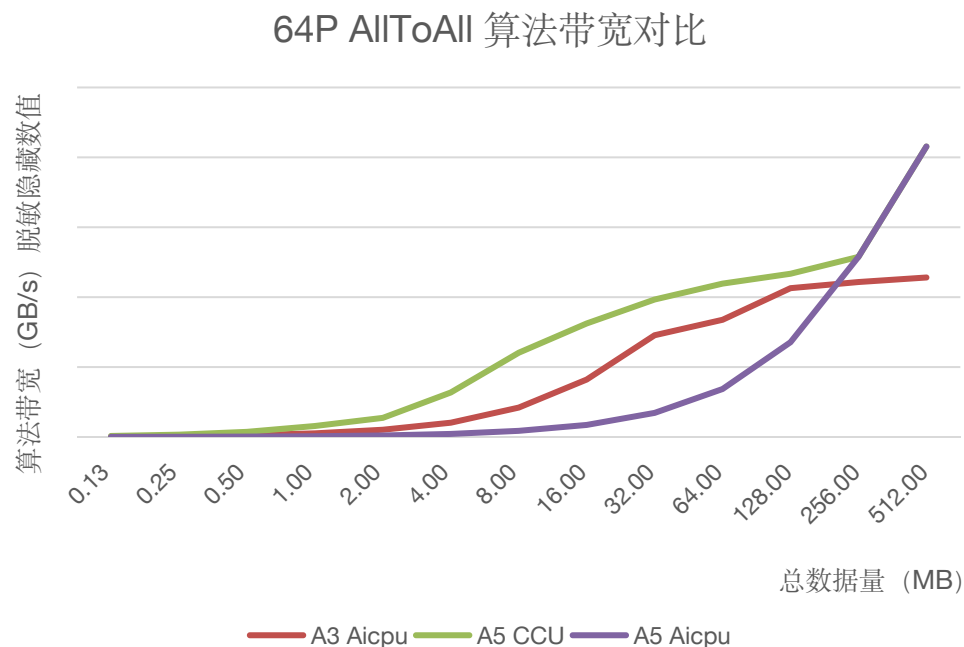
- 这个算子已经开源，https://gitcode.com/cann/ops-transformer/tree/master/mc2/moe_ep_dispatch
- 盘古模型正在适配该算子，暂时无法提供使用 moe_ep_dispatch 算子 vs 使用 AllToAllV算子的整网性能对比。

■ 硬件亲和：使能 CCU 通信，提升 AllToAll 开箱性能

昇腾 950 已经公开了 NPU 架构白皮书，增加了一个专门用于通信的加速芯片：CCU (Collective Communication Unit)

昇腾 950 有3种不同的通信算子展开方式（通信算子拆包，按业务流程给网卡下发通信指令）：AICPU、AIV、CCU

展开方式	优点	缺点	适用场景
AICPU (默认)	资源充足：支持6个通信域的算子并发执行	执行速度慢（通信数据量很小则成为瓶颈）	训练、Prefill 数据量较大的场景
AIV	load/ store 内存语义访问，协议栈简单	每次搬运的数据量小；占用计算核的算力	Decode 数据量小 & 时延要求严苛的场景
CCU	专用硬件，处理速度快	片上缓存/计算资源紧张	仅提供给性能要求最高的通信域

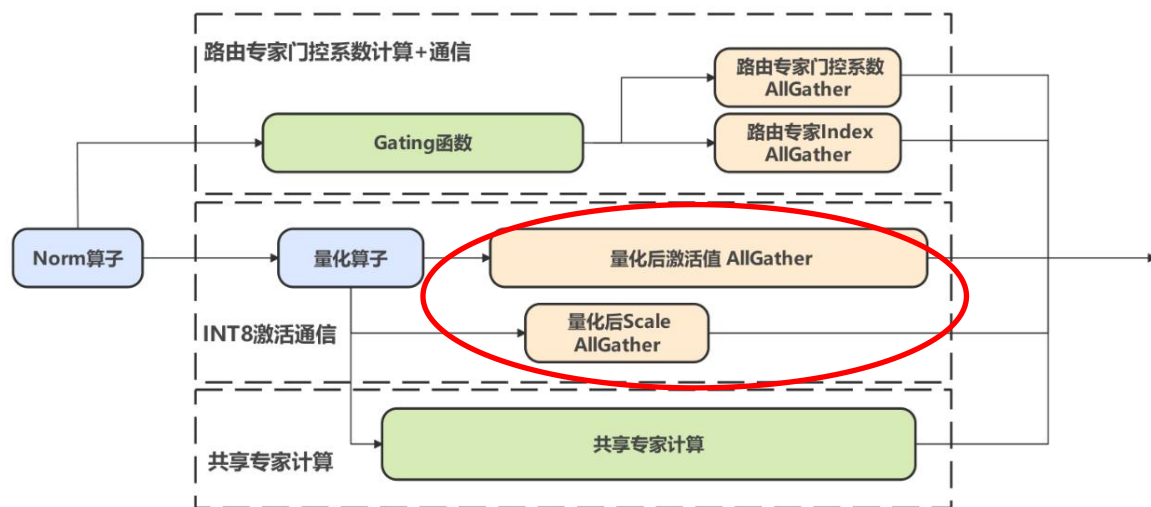


根据盘古模型训练的经验，EP AllToAll 占有所有通信时间的85%+，建议这个通信域使用CCU模式，其他通信域使用Aicpu模式

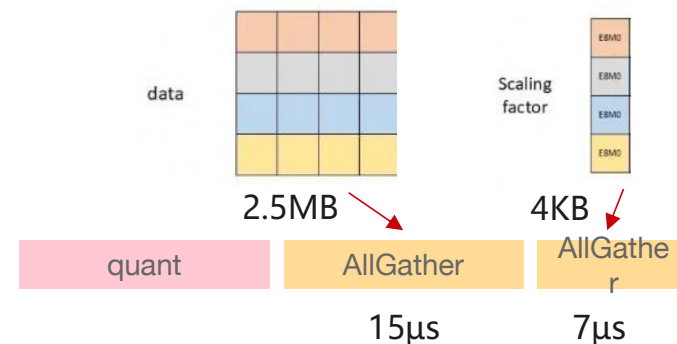
模型亲和：针对盘古模型量化+通信的时序，在推理Decode场景开发自定义算子，加速推理TPoT

盘古在 MoE 计算前逐 token 量化（未来演进到逐 block 量化，例如MXFP8），此时Tensor中不同位置对应不同缩放因子。需要向专家同时传输量化结果和缩放因子。

缩放因子的数据量小，传输静态开销大，导致传输效率低。以8P场景AllGather举例：



Before: 两个AllGather，分别传输量化结果和量化因子



After: 一个AllGatherBatch，同时传输量化结果和量化因子



CANN 在26年中开放了通信底层能力抽象 HCOMM (<https://gitcode.com/cann/hcomm>)，让开发者可以实现自定义的通信

原语。

■ 小结: MoE 阶段 AllToAll通信加速 take away

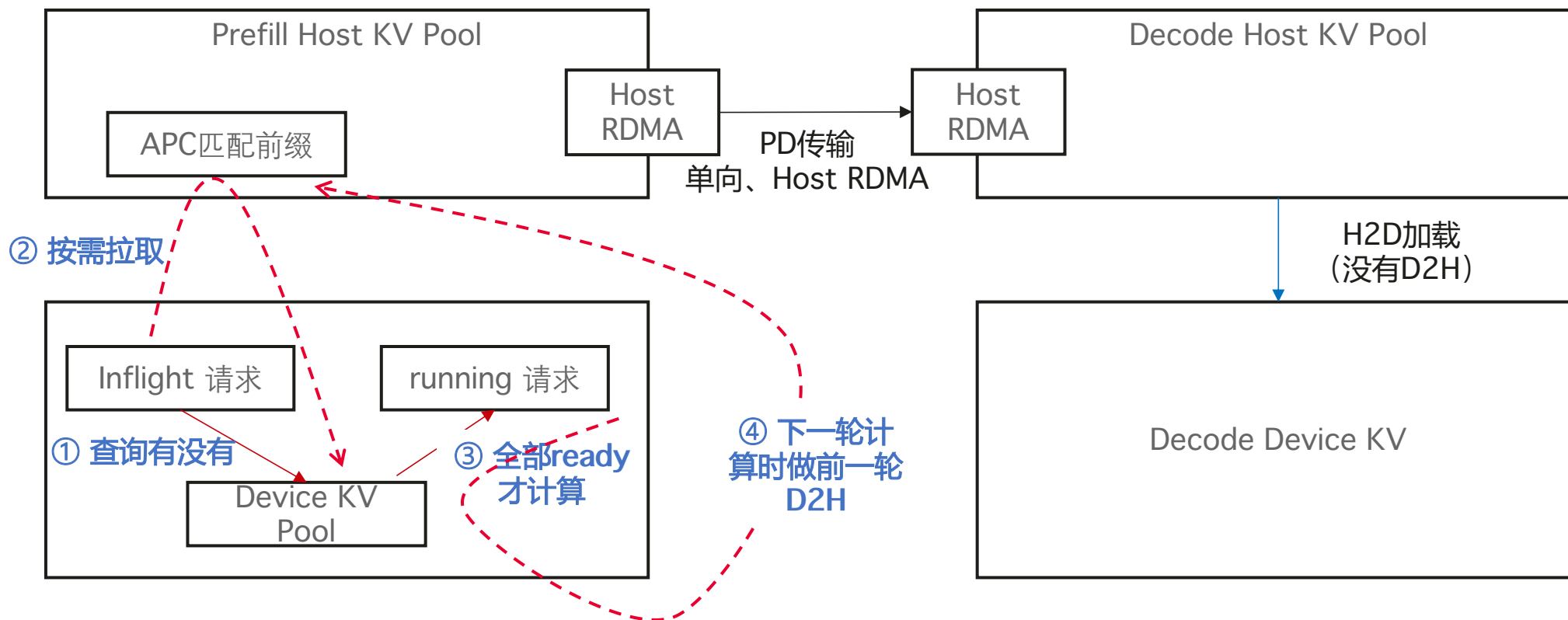
业务场景	优化目标	优化手段
训练	极致性能	使用 moe_ep_dispatch 算子替代 AllToAllV , 实现 1) NPU驱动通信 , 算子零断流: top k 计算之后立刻开始通信 2) 通信和计算掩盖 : Dispatch通信过程中, host 收集各 rank 的 token 数量, 下发GMM算子。
	开箱性能	使用 CCU 模式执行AllToAllV 算子 , 实现 1) 无需改动模型脚本, 只需要在创建通信域时传入一个参数 2) AllToAll的通信性能提升 30%
推理	TTFT (Prefill)	同训练场景
	TPOT (Decode)	使用自定义算子 AllGatherBatch , 实现 1) 量化结果和量化因子一起传输, 从两个算子减少为一个算子 2) TPOT减少300~400us

03

硬件软件双管齐下， 加速H2D KV Cache加载

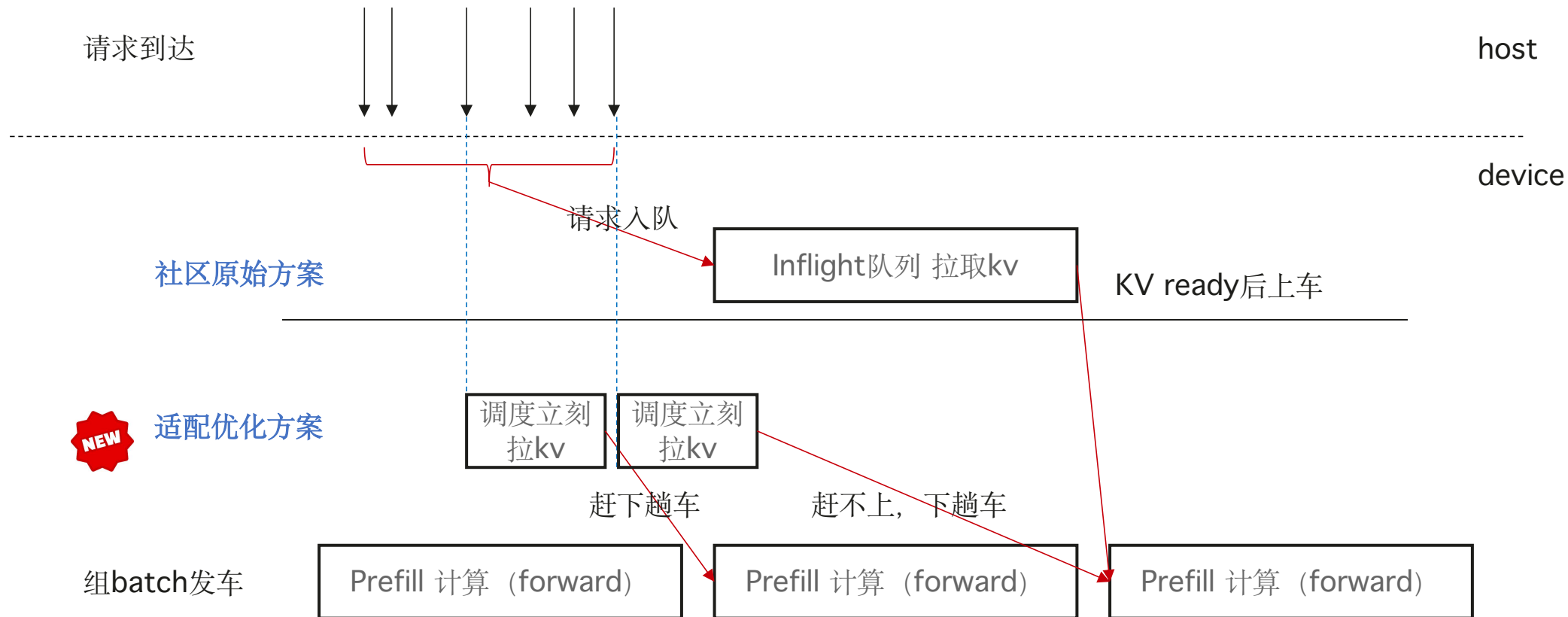


VLLM 经提供 KV Offload 特性，按请求粒度进行卸载



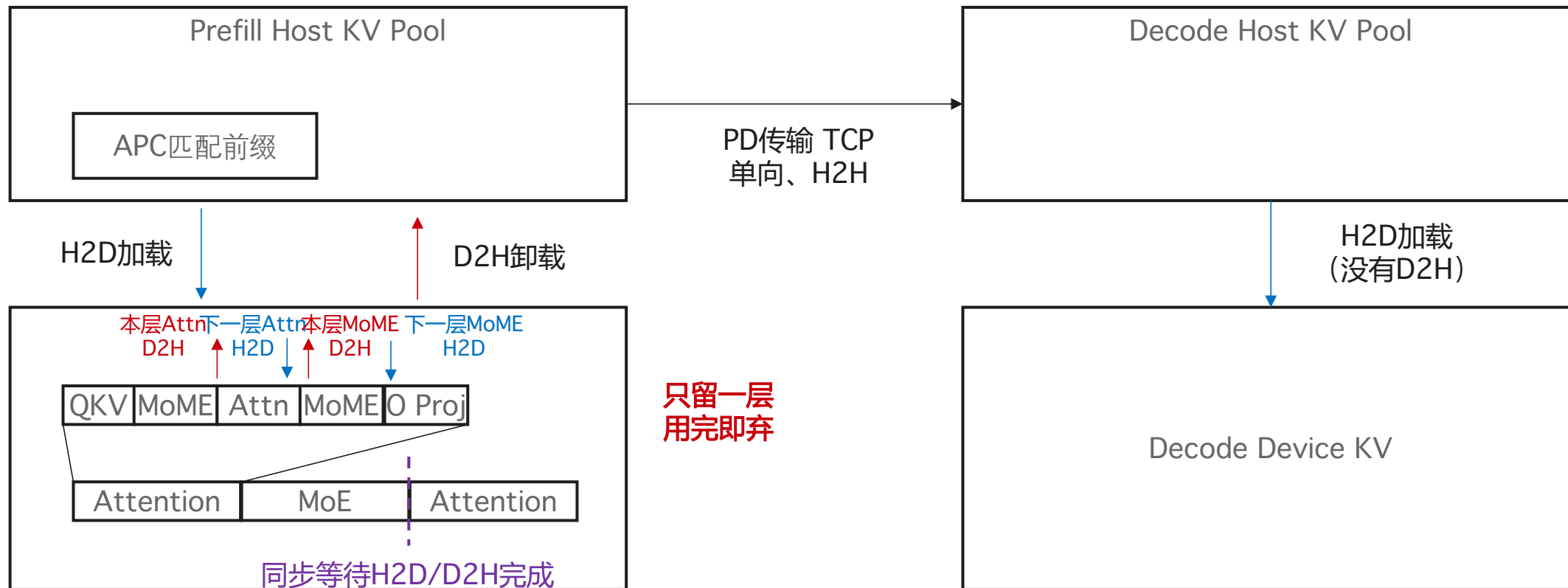
- 加载时序：请求达到后进入**waiting**队列，前缀匹配后进入**inflight**队列，KV Cache传输完成之后进入**running**队列
- 卸载时序：下一个step进行kv Cache卸载

VLLM 的调度策略会导致 TTFT 增加一个 forward step 时间 (300ms~100ms)



对于绝大部分业务，需要将TTFT压缩到2秒内。社区现有方案难以达到这个要求。我们提出了新的调度方案。

■ 针对超长上下文，Omni Infer 提出按layer粒度加载



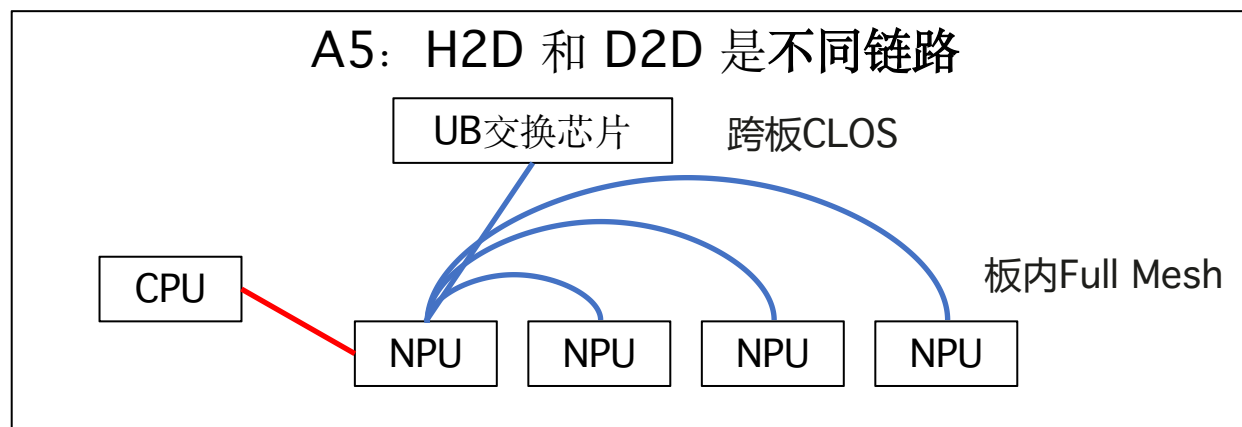
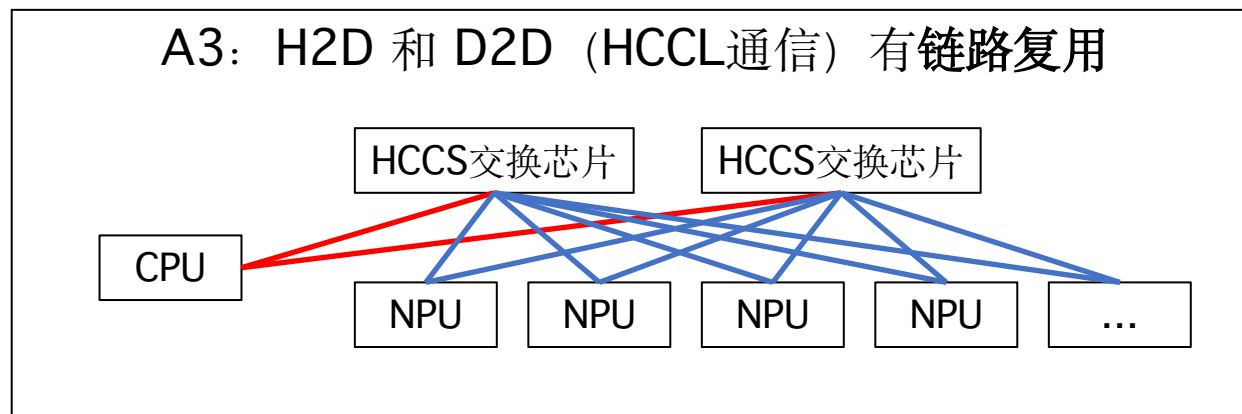
已经开源: <https://gitee.com/omniai/omni-cache/>

收益: 单个 512K 上下文请求 KV Cache 占用的 HBM 空间从 12GB 降低到 0.67GB

代价: APC命中的序列很长, 剩余要计算的新序列比价短 (<5k), 则会出现CPU忙于KV Cache处理, 导致算子下发 bound

■ 硬件优化: A5 提供独立的 H2D 链路, 应对日益重要的 Host to Device 通信

业务	H2D/D2H业务
训练	Check Point 卸载、加载
	激活值卸载、加载, 减少重计算
推理	权重加载
	KV Cache 卸载、加载 NEW
	Engrame N-Gram 词嵌入向量 NEW



A3 建议配置H2D流量和D2D流量的优先级, 参考开源: ALLM Ascend Ai_QoS

https://docs.vllm.ai/projects/ascend/en/main/user_guide/feature_guide/Ai_QoS_introduction_en.html

■ 小结: Attention 阶段 H2D 通信加速 take away

业务场景	优化目标	优化手段
推理	最小化 TTFT	卸载粒度: KV Cache 按照 request 粒度进行加载和卸载 加载时序: 请求到达之后立刻开始 KV Cache 加载, 与其他 request 的计算实现并行
	最小化 HBM 空间	卸载粒度: KV Cache 按照 layer 粒度进行加载和卸载 加载时序: 当前 layer 计算过程中加载下一个 layer 的 KV Cache, 与当前 layer 的 MoE 计算并行
	优化端到端性能	当代硬件 (A3): 配置 H2D 流量和 D2D 流量的优先级 下一代硬件 (A5): 不需要任何配置

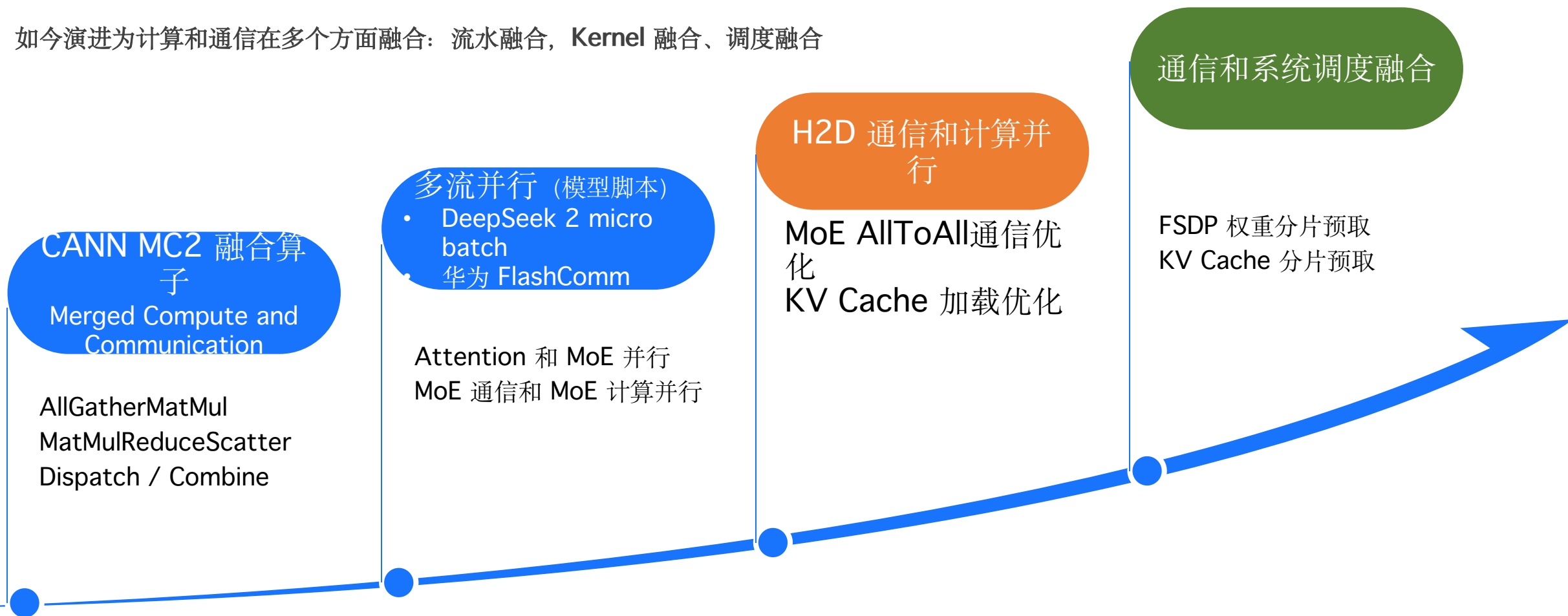
04

总结和展望： 减少未被掩盖的通信时间

■ 通信优化的目标： 计算掩盖通信， 实现算力发挥到极致

过去的系统架构是“算子库”+“通信库”拼装调用

如今演进为计算和通信在多个方面融合：流水融合， Kernel 融合、调度融合



QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会